

3교시: 인프라 엔지니어가 MSA에서 알아야 할 것

수업 목표

- 서비스 목록, 포트, 프로토콜, 의존성, 설정, 헬스체크, 로그 위치를 MSA 운영 문제로 설명한다.
- 서비스 경계, 네트워크, 설정, 로그, health 중 최소 두 가지 evidence로 정상 상태를 판단한다.
- 장애가 나면 재현, 관찰, 가설, 수정, 재확인 순서로 기록한다.

선행 지식

이미 알고 있어야 할 것	오늘 다시 연결할 것
Docker image/container	MSA의 서비스별 실행 단위
Docker Compose service	서비스 토폴로지와 의존성
port binding, service name DNS	내부 통신과 외부 진입점 구분
logs/status evidence	분산 로그와 장애 분석

50분 흐름

시간	활동	학습 초점	학생 산출
0-5분	문제 상황 제시	왜 단일 앱보다 운영이 어려워지는지 본다.	오늘의 질문 1개
5-15분	핵심 개념 설명	인프라 엔지니어가 MSA에서 알아야 할 것의 운영 의미를 정리한다.	용어 note
15-25분	구조/공식 기준 확인	service, dependency, health, logs를 공식 용어와 연결한다.	판단 표
25-40분	실습 또는 데모	Compose 상태와 로그로 실제 증거를 확인한다.	command evidence
40-47분	장애/비용/보안 점검	운영 위험과 복구 기준을 정리한다.	risk note
47-50분	다음 교시 연결	다음 service boundary로 확장한다.	handoff 문장

시작 상황

서비스가 하나일 때는 container가 켜졌는가 , port가 열렸는가 , HTTP 응답이 오는가 를 보면 대부분의 상태를 파악할 수 있다. MSA에서는 같은 질문을 서비스마다 반복해야 하고, 한 서비스의 정상 상태가 전체 사용자 경험의 정상 상태를 보장하지 않는다.

오늘의 주제인 인프라 엔지니어가 MSA에서 알아야 할 것 은 이 복잡도를 줄이기 위한 관찰 기준을 만든다. 인프라/DevOps 엔지니어는 내부 구현을 모두 알 필요는 없지만, 어떤 서비스가 어떤 주소로 누구에게 요청하고, 어떤 설정을 필요로 하며, 어디에서 실패 증거를 남기는지는 알아야 한다.

Generated Visual: 3교시 전용 인포그래픽

인프라 엔지니어가 MSA에서 알아야 할 운영 계약

개발팀과 인프라팀의 약속을 문서화하여 안정적인 배포와 운영을 보장합니다.



운영 계약

서비스가 운영 환경에서 안정적으로 동작하기 위해 개발팀이 제공해야 하는 정보와 약속입니다.

서비스 목록 (서비스명)	이미지 (컨테이너 이미지)	포트 (서비스 포트)	프로토콜 (통신 프로토콜)	환경변수 (필수 환경변수)	의존성 (외부 의존 서비스)	health (헬스 체크 엔드포인트)	logs (로그 확인 명령어)	개발팀 전달사항 (운영 담당자/연락처)
order-service	order-service:1.0.0	8080	HTTP	DB_HOST DB_USER DB_PASSWORD LOG_LEVEL	mysql (redis)	GET /actuator/health	kubectl logs -f -app=order-service	이주연 (Order팀) 010-1111-2222
product-service	product-service:1.0.0	8080	HTTP	DB_HOST DB_USER DB_PASSWORD LOG_LEVEL	mysql (warehouse-service)	GET /actuator/health	kubectl logs -f -app=product-service	박민수 (Product팀) 010-2222-3333
user-service	user-service:1.0.0	8080	HTTP	DB_HOST DB_USER DB_PASSWORD JWT_SECRET LOG_LEVEL	mysql (redis)	GET /actuator/health	kubectl logs -f -app=user-service	최수빈 (User팀) 010-3333-4444
payment-service	payment-service:1.0.0	9090	HTTP	PG_URL PG_KEY LOG_LEVEL	external-pay-gateway (redis)	GET /actuator/health	kubectl logs -f -app=payment-service	정현우 (Payment팀) 010-4444-5555
delivery-service	delivery-service:1.0.0	8081	HTTP	MAPS_API_KEY LOG_LEVEL	maps-api (notification-service)	GET /actuator/health	kubectl logs -f -app=delivery-service	김도윤 (Delivery팀) 010-5555-6666

운영 계약 체크리스트

- ✓ 서비스명과 컨테이너 이미지 태그가 명확히 정의되었는가?
- ✓ 포트와 프로토콜(HTTP/HTTPS/gRPC 등)이 명시되었는가?
- ✓ 필수 환경변수가 빠짐없이 정의되었는가?
- ✓ 외부 의존 서비스와 의존 방향이 정확한가?
- ✓ 헬스 체크 엔드포인트가 구현되어 있고 응답 기준이 합의되었는가?
- ✓ 로그 확인 명령어가 제공되었는가?
- ✓ 운영 담당자와 비상 연락처가 명시되었는가?

개발팀 전달사항 (운영에 필수!)

- 배포 문서
빌드/배포 절차, 마이그레이션(필요시), 롤백 절차
- 설정 정보
필수 환경변수 목록, 기본값, 예시
- 장애 대응 가이드
자주 발생하는 장애와 복구 방법
- 모니터링 지표
핵심 비즈니스 지표, 애플리케이션 지표

왜 운영 계약이 중요한가?

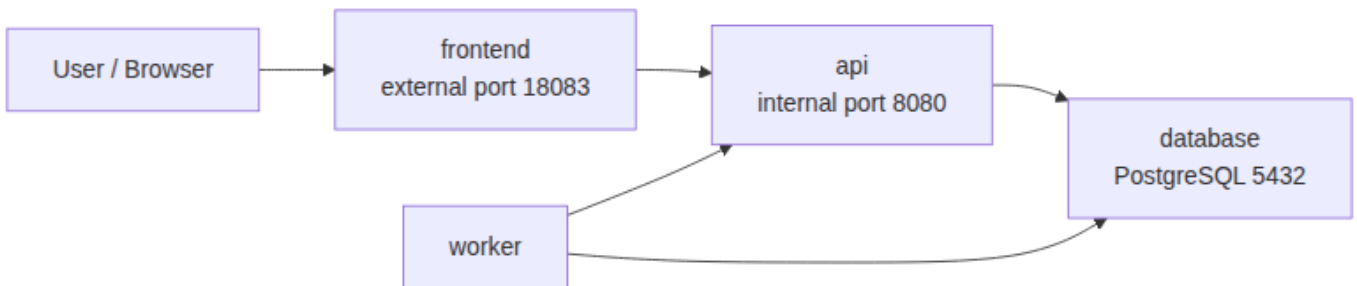
- ✓ 배포 속도 향상: 필요한 정보가 명확하여 반복 질문 감소
- ✓ 장애 대응 시간 단축: 로그 위치, 헬스 체크로 빠른 원인 파악
- ✓ 책임 소재 명확: 연락처와 소유권으로 의사결정 단순화
- ✓ 표준화된 운영: 모든 서비스가 동일한 기준으로 관리
- ✓ 지식 공유 용이: 문서화된 정보로 팀 변경 시에도 안정적 운영

TIP 운영 계약은 코드 변경 시 함께 업데이트하세요. (새로운 환경변수 추가, 포트 변경, 의존성 변경 등)

개발 완료 → 운영 계약 업데이트 → 인프라 검토 → 배포 → 운영

이 이미지는 Week 3 Day 1 3교시 인프라 엔지니어가 MSA에서 알아야 할 것 의 핵심 개념을 세션 전용으로 시각화한 자료다. 다른 교시와 같은 그림을 재사용하지 않고, 이 세션에서 확인할 운영 evidence와 판단 기준에 맞춰 읽는다.

Visual 1: 서비스 토폴로지 읽기



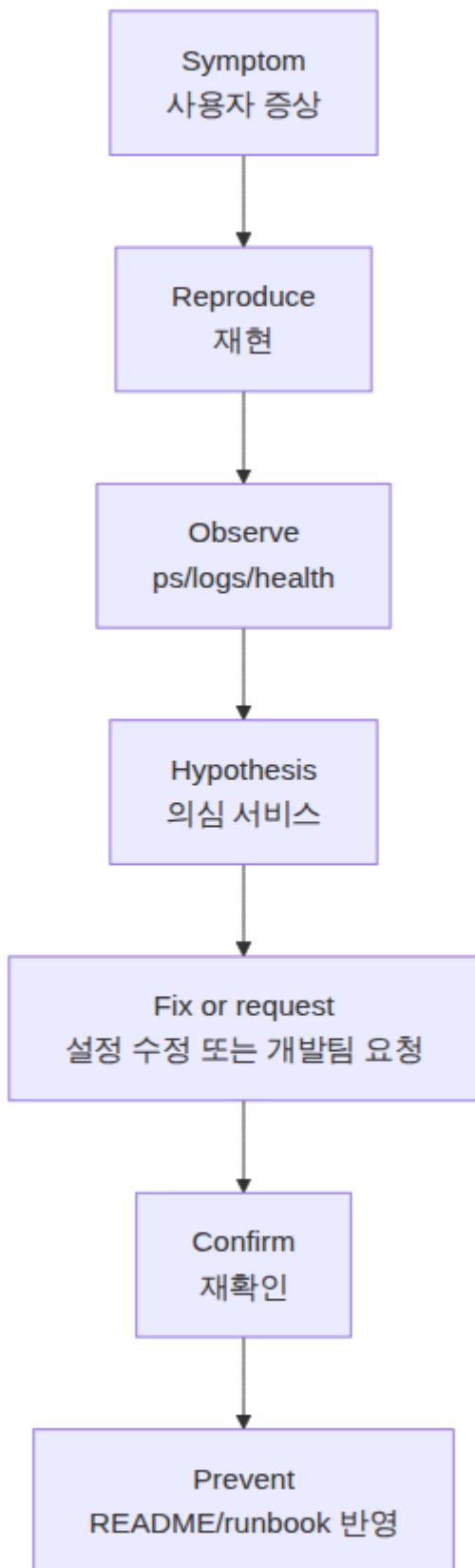
읽는 순서: 사용자는 frontend로 들어오고, frontend는 api로 요청을 넘긴다. api와 worker는 database에 의존한다. 이 그림에서 중요한 것은 박스 개수가 아니라, 장애가 어디에서 어디로 전파될 수 있는지 보는 것이다.

Visual 2: 운영 판단 표

확인 대상	정상 evidence	실패 증상	다음 행동
frontend	browser 접속, HTTP 200	화면은 뜨지만 API 데이터 없음	proxy/API URL 확인
api	/health, /api/status 응답	5xx, timeout, DB unavailable	env, DB host, logs 확인
worker	주기적 처리 로그	처리 정지, 반복 실패	dependency와 retry 주기 확인
database	healthcheck healthy	connection refused	volume, credential, readiness 확인

이 표는 기능 목록이 아니라 장애 분석 순서다. 운영자는 사용자가 본 증상에서 시작해 어느 서비스 evidence를 먼저 볼지 정해야 한다.

Visual 3: 장애 분석 흐름



이 흐름은 Week 1의 RCA와 같다. 달라진 점은 관찰 대상이 하나의 process가 아니라 여러 service의 조합이라는 점이다.

실습 기준

repository root에서 시작한다. Day 1 이후 모든 교시는 같은 실습 앱을 기준으로 진행한다.

```
cd week3/day1/labs/msa-demo
docker compose ps
docker compose logs --tail=40 api
docker compose logs --tail=40 worker
curl -s http://localhost:18083/api/status
```

기대 결과는 서비스 상태가 모두 `running` 또는 `healthy` 에 가깝고, `/api/status` 가 frontend를 통해 api 상태를 JSON으로 돌려주는 것이다. 실패하면 실패 자체를 evidence로 남긴다.

DevOps 원칙 연결

관점	오늘의 연결
비용 절감	로컬 Compose로 topology를 먼저 검증해 cloud 리소스 생성 전 오류를 줄인다.
개발/배포 효율	서비스별 실행 계약을 문서화해 개발팀과 운영팀의 왕복 질문을 줄인다.
관리 효율	health, logs, dependency map을 표준화해 장애 triage 시간을 줄인다.

흔한 오해

오해	바로잡기
MSA는 서비스를 많이 나누면 자동으로 좋아진다.	서비스 경계가 운영 비용보다 큰 이익을 줄 때 선택한다.
frontend가 보이면 전체 시스템이 정상이다.	API나 DB 장애가 숨어 있을 수 있으므로 service별 health를 봐야 한다.
depends_on이면 DB 준비까지 보장된다.	실행 순서와 readiness는 다르다. healthcheck와 retry가 필요하다.
로그를 보면 다 알 수 있다.	여러 서비스 로그를 연결할 request id/correlation id가 없으면 원인 추적이 느려진다.

Evidence Note 양식

```
# Week 3 Evidence - 인프라 엔지니어가 MSA에서 알아야 할 것

## Service checked
- Service:
- Command:
- Expected:
- Actual:

## Dependency
- Upstream:
- Downstream:
- Config/env used:

## Failure or risk
- Symptom:
- Observed evidence:
```

- Fix or request:
- Recheck result:

평가 기준

기준	통과 조건
개념 이해	인프라 엔지니어가 MSA에서 알아야 할 것을 서비스 경계/의존성/운영 증거 중 하나와 연결했다.
실행 증거	Compose 상태, 로그, HTTP 응답 중 하나 이상을 기록했다.
장애 분석	증상과 의심 서비스를 구분했다.
협업 기준	개발팀에 요청할 정보 또는 운영 문서에 남길 정보를 적었다.
다음 연결	Kubernetes에서 어떤 리소스로 이어질지 한 문장으로 설명했다.

공식/학술 근거 링크

- AWS Microservices on AWS, <https://docs.aws.amazon.com/whitepapers/latest/microservices-on-aws/microservices.html> - MSA를 여러 서비스와 API 기반 구조로 설명하는 공식 기준이다.
- Docker Compose Docs, <https://docs.docker.com/compose/> - 여러 service를 파일로 실행하고 관리하는 기준이다.
- Google SRE Book: Addressing Cascading Failures, <https://sre.google/sre-book/addressing-cascading-failures/> - 한 서비스 장애가 다른 서비스로 퍼질 수 있음을 설명하는 근거다.

다음 연결

다음 교시는 같은 실습 앱에서 다른 service boundary를 본다. 오늘 만든 evidence note는 서비스 목록, 의존성, health, logs를 누적하는 운영 문서가 된다.